

Classifications of Multiprocessor Systems



1. Loosely coupled or distributed multiprocessor (Cluster)

- Independent systems working together.
- **Each processor has its own memory** and input/output channels.

2. Functionally specialized processors

- One main **processor controls** everything.
- Other processors perform specific tasks and support the main processor.

3. Tightly coupled multiprocessor

- Processors **share the same memory**.
- They work under one operating system.

Synchronization Granularity and Processes



1. Fine Grain:

Parallel tasks in a single instruction stream, sync interval <20 instructions.

2. Medium Grain:

Multitasking or parallel processing in one app, sync interval 20-200 instructions.

3. Coarse Grain:

Concurrent processes in multiprogramming, sync interval 200-2000 instructions.

4. Very Coarse Grain:

Distributed processing across network nodes, sync interval 2000-1M instructions.

5. Independent:

Unrelated processes, no sync needed.

concurrent
(eş zamanlı)

Independent Parallelism



- No need for synchronization between processes.
- Each process operates as an independent application or job.
- Typically used in time-sharing systems.
- **Each user runs a specific application.**
- **Multiprocessor systems provide similar functionality to multiprogrammed uniprocessors.**
- **With multiple processors, users experience more shorter average response times.**

Coarse and Very Coarse Grained Parallelism Summary

- Synchronization happens, but at a high level.
- Managed as concurrent processes on a multiprogrammed uniprocessor.
- Can be used on multiprocessors with minimal or no changes to user software

Medium-Grained Parallelism Summary

- A single application uses multiple threads within one process.
- The programmer must define parallel parts explicitly.
- Threads need strong coordination and interaction.
- Scheduling one thread can impact the whole application's performance.

Fine-Grained Parallelism Summary

- More complex than thread-based parallelism.
- Involves specialized techniques and various approaches.

Design Issues Summary

- Multiprocessor scheduling involves three key issues:
 1. **Dispatching processes**
 2. **Using multiprogramming on processors**
 3. **Assigning processes to processors**
- The approach depends on application granularity and available processors.



Assignment of Processes to Processors

- Both **dynamic** and **static** methods assign processes to processors.
- Two main approaches:
 1. **Master/Slave**
 2. **Peer**

Master/Slave Architecture



- Key kernel functions always run on a specific processor (master).
- Master is responsible for scheduling.
- Slave sends service request to the master.
- Simple design, minimal changes needed for multiprogramming.
- Conflict resolution is easier since one processor manages memory and I/O.

Peer Architecture

- The kernel can run on any processor.
- Each processor schedules processes independently.

Complication:

- The operating system must prevent two processors from selecting the same process and ensure no processes are lost from the queue.

Process Scheduling

- In traditional multiprocessor systems, processes are not tied to specific processors.
- All processors share a single queue.
- If priorities are used, multiple queues feed into the common processor pool.
- The system works like a multi-server queuing model.

Thread Scheduling

- Thread execution is separate from the process definition.
- Application can run multiple threads together in the same address space.
- On uniprocessors, threads improve I/O and processing overlap.
- On a multiprocessor, threads enable real parallelism.
- Proper thread management greatly impacts performance.



Approaches to Thread Scheduling

Four approaches for multiprocessor thread scheduling and processor assignment are ;

1. Load Sharing:

Processes are not assigned to specific processors.

2. Gang Scheduling:

Related threads run on multiple processors at the same time.

3. Dedicated Processor Assignment:

Threads are assigned to specific processors for implicit scheduling.

4. Dynamic Scheduling:

The number of threads can change during execution.



Load Sharing

- Simplest method, adapted from uniprocessor systems.
- Versions:
 1. FCFS (First-Come-First-Served)
 2. Smallest number of threads first
 3. Preemptive smallest number of threads first

Advantages:

- The load is evenly(eşit) distributed across processors, ensuring that none of them remain idle(bos).
- No need for a centralized scheduler.
- Global queue can be managed and organized any of the schemas.



Disadvantages of Load Sharing

- Central queue needs mutual exclusion, leading to bottlenecks.
- Preemptive threads may not run on the same processor, lowering caching efficiency.
- Hard for all program threads to run simultaneously.
- Frequent process switching harms performance.

Gang Scheduling

- Threads of a process are scheduled simultaneously on multiple processors.
- Useful for medium to fine-grained parallel applications where performance drops if some parts are idle.
- Also helps any parallel application.

Benefits

- Less synchronization blocking and fewer process switches, improving performance.
- Reduced scheduling overhead.

Dedicated Processor Assignment

- Each thread is assigned to a specific processor, and the processor remains dedicated until the thread finishes.
- If a thread is waiting for I/O or synchronization, its processor stays idle.
- No multiprocessing is done on processors.
- **Advantage:** In highly parallel systems, avoiding process switching improves performance significantly.

Dynamic Scheduling

- The number of threads can change during execution to improve utilization.
- The operating system mainly handles processor allocation.
- Both the OS and application decide scheduling.
- It's better than gang scheduling or dedicated assignment for some applications.

Cache Sharing

1. Cooperative Resource Sharing

- Multiple threads share the same main memory locations.
- Examples: Multithreaded applications, producer-consumer threads.

2. Resource Contention(competition)

- Threads on adjacent cores compete for cache space.
- Uneven cache allocation reduces performance.
- Contention-aware scheduling aims to improve cache efficiency by balancing thread allocation.

Real-Time Systems

- The operating system, especially the scheduler, is crucial.
- The system's correctness depends on both results and timing.
- Tasks control or respond to real-world events in real time.
- These events happens in "real time". Tasks must be able to keep up with them.

Examples

- Laboratory control, industrial process control, robotics, air traffic control, telecommunications, military systems.



Hard Real-Time Task

- Must **definitely** ! meet its deadline.
- Missing the deadline can cause serious damage or system failure.



Soft Real-Time Task

- Has a preferred deadline, but it's not **mandatory**.
- Completing the task after the deadline still makes sense.



Periodic Tasks

- Tasks occur at regular intervals.
- Happens once per period **T**.
- The interval between tasks is exactly **T** units.



Aperiodic Tasks

- Has a **deadline** to start or finish.
- May have constraints on both start and finish time.